

Laporan Jobsheet 12 /Adiatma Wahyudi/TI 1C

NIM:254107020212

12.2.3 Pertanyaan Percobaan 1 (Halaman 7)

1. Perbedaan Struktur dan Mekanisme Traversal Antara SLL dan DLL

Struktur: Single Linked List (SLL) hanya memiliki satu pointer penghubung pada setiap node (next) yang mengarah ke node berikutnya. Sedangkan **Double Linked List (DLL)** memiliki dua pointer pada setiap node, yaitu next (mengarah ke node berikutnya) dan prev (mengarah ke node sebelumnya).

Mekanisme Traversal: Pada SLL, penelusuran (traversal) hanya bisa dilakukan searah (*forward*), yaitu dari head menuju ke depan. Pada **DLL**, penelusuran bisa dilakukan secara dua arah, yaitu maju dari head ke depan maupun mundur dari tail ke belakang.

2. Fungsi Atribut next dan prev pada Class Node

Atribut next: Berfungsi untuk menyimpan referensi/alamat memori dari node selanjutnya. Digunakan untuk navigasi maju saat traversal dan menghubungkan node saat menyisipkan atau menghapus data.

Atribut prev: Berfungsi untuk menyimpan referensi/alamat memori dari node sebelumnya. Digunakan untuk navigasi mundur saat traversal dan menjaga agar rantai pointer ke belakang tidak terputus saat terjadi manipulasi node.

3. Fungsi Konstruktor Terhadap Kondisi Awal Linked List

Konstruktor pada class DoubleLinkedList berfungsi untuk menginisialisasi keadaan awal list ketika objek pertama kali dibuat. Konstruktor ini mengatur atribut head dan tail menjadi null, yang menandakan bahwa **linked list dalam keadaan kosong** (belum memiliki node sama sekali).

4. Kenapa pas linked list kosong, head sama tail nunjuk ke node yang sama?

Bayangin kamu punya kotak kosong, terus kamu masukin satu barang. Karena barangnya **cuma satu-satunya**, otomatis barang itu jadi barang pertama sekaligus barang terakhir yang ada di dalam kotak. Makanya, head (awal) dan tail (akhir) terpaksa harus menunjuk ke node yang sama.

5. Modifikasi method print() biar muncul tulisan kalau kosong

Sebenarnya di contoh langkah 11 sudah ada kodenya. Biar makin rapi, kodenya bisa dibuat seperti ini:

Java

```
public void print() {  
    if (isEmpty()) {  
        System.out.println("Linked List masih kosong.");  
        return;  
    }  
    Node current = head;  
    while (current != null) {  
        current.data.tampil();  
        System.out.println("-----");  
        current = current.next;  
    }  
}
```

6. Tambahan method printReverse() (Cetak Terbalik)

Tinggal kita balik aja logikanya. Mulai jalannya dari tail (paling belakang), lalu gesernya mundur pakai prev.

Java

```
public void printReverse() {  
    if (isEmpty()) {  
        System.out.println("Linked List masih kosong.");  
        return;  
    }  
    Node current = tail;  
    while (current != null) {  
        current.data.tampil();  
        System.out.println("-----");  
        current = current.prev;  
    }  
}
```

```
}  
}
```

Pertanyaan Percobaan 2(hal 9)

1. Maksud dari dua baris kode di `removeFirst()` ini apa?

Java

```
head = head.next;
```

```
head.prev = null;
```

head = head.next; Artinya kita mindahin status "pemimpin" (head) ke node kedua. Node pertama yang lama jadi tersingkirkan.

head.prev = null; Node kedua yang sekarang naik pangkat jadi head baru, penunjuk belakangnya (prev) harus diputus jadi null. Kenapa? Biar dia nggak nginget-nginget lagi node pertama yang udah kita hapus tadi, jadi node lama itu bisa langsung dibuang dari memori.

2. Modifikasi `removeFirst()` dan `removeLast()` biar pamer data yang dihapus

Biar programnya ngasih tahu data siapa yang baru aja dihapus, kita tampung dulu datanya ke variabel sebelum dihilangkan.

Modifikasi yang dilakukan:

Java

```
public void removeFirst() {  
    if (isEmpty()) {  
        System.out.println("Linked List kosong.");  
        return;  
    }  
}
```

```

    }

    Mahasiswa dataDihapus = head.data;

    if (head == tail) {
        head = tail = null;
    } else {
        head = head.next;
        head.prev = null;
    }

    System.out.println("Data berhasil dihapus:");
    dataDihapus.tampil();
}

```

Ubah removeLast() jadi gini:

Java

```

public void removeLast() {
    if (isEmpty()) {
        System.out.println("Linked List kosong.");
        return;
    }
}

```

```

    Mahasiswa dataDihapus = tail.data;

```

```

    if (head == tail) {
        head = tail = null;
    } else {
        tail = tail.prev;
        tail.next = null;
    }
}

```

```
System.out.println("Data berhasil dihapus:");  
dataDihapus.tampil();  
}
```